

**A PROJECT REPORT**  
**ON**  
**INSIDER THREAT DETECTION USING**  
**ELASTIC NET AND RANDOM FOREST**

Submitted in partial fulfilment of the requirements for the Course

**MACHINE LEARNING**  
**BACHELOR OF TECHNOLOGY**

A.Y.: 2026-27

**By**

CHAVA NOHITHA – 2520030130

G. LAASYA – 2520030118

KALLU SHRESTA PRIYA – 2520030127

**Under the Esteemed Guidance of**

Gaddam Ravindra Babu

Assistant Professor, CSE

**Department of Computer Science and Engineering**

Koneru Lakshmaiah Education Foundation (Deemed to be University)

Bachupally–Gandimaisamma Road, Bowrampet, Hyderabad, Telangana – 500 043

## Abstract

Insider threats are security risks that originate from people who already hold authorised access to an organisation's systems, such as employees, contractors and business partners. Because these users operate with legitimate credentials, their harmful actions, which may include data theft, misuse of privileges or sabotage, often resemble ordinary work and are difficult to separate from normal behaviour. Manual monitoring cannot keep pace with the very large volume of logon, file, device, email and web activity generated in a modern enterprise, and fixed rule-based alerts miss unusual combinations of otherwise normal actions.

This project develops a machine learning system that classifies user activity as normal or potentially malicious. Raw activity records are aggregated into behavioural features such as login count, after-hours activity, file access count, USB usage, external email activity and web activity. Elastic Net, a regularised model that combines the L1 penalty of Lasso with the L2 penalty of Ridge Regression, is used to select the most informative features: the L1 component shrinks the coefficients of uninformative features to exactly zero, while the L2 component keeps the coefficients of correlated features stable. A Random Forest classifier, an ensemble of decision trees that can model non-linear relationships and interactions between behaviours, is then trained on the selected features to predict whether activity is normal or a potential insider threat.

The report describes the complete workflow: data understanding and exploratory analysis, cleaning, time-based feature engineering, scaling, leakage-safe train-test splitting, class-imbalance handling, model training and tuning by cross-validation, and evaluation with accuracy, precision, recall, F1-score, ROC-AUC and PR-AUC. Because insider-threat data are heavily imbalanced, recall and PR-AUC receive particular attention. An interactive web dashboard, named Insider Guard, accompanies the model and provides sections for user activity, threat prediction, alerts, analytics, model evaluation, monitoring and reports. The expected practical use is an early-warning tool that helps security analysts prioritise suspicious activity and reduce the risk of data loss.

**Keywords:** insider threat detection; Elastic Net; Random Forest; feature selection; class imbalance; CERT dataset; user behaviour analytics; cybersecurity

# **Chapter 1: Introduction**

## **1.1 Background**

Organisations depend on digital systems to store and exchange sensitive information, and most security investment is directed at external attackers. However, a user who already has legitimate access can cause equal or greater harm, because the usual perimeter defences do not stop them. An insider threat occurs when an employee or other authorised user misuses access to company resources, intentionally or unintentionally. Typical forms include copying confidential files to removable media, sending data to external mailboxes, working at unusual hours to avoid attention, and probing systems beyond the user's role.

Traditional rule-based monitoring raises an alert when a fixed threshold is crossed, such as more than a set number of file accesses in a day. Such rules are easy to evade and generate many false alarms because the right threshold differs from user to user and role to role. Machine learning can instead learn from historical activity which patterns and combinations of behaviour are associated with normal and suspicious users, and can classify new activity within seconds.

## **1.2 Need for Insider Threat Detection**

Insider activity is difficult to distinguish from normal activity, and the amount of data produced by users is too large for manual review. A suspicious pattern may be missed simply because it is buried among millions of routine events. Automated classification supports earlier identification of potentially suspicious behaviour, allowing analysts to investigate before data are lost. Feature selection is equally important: with many behavioural variables, irrelevant or redundant features add noise, slow training and reduce the clarity of the result.

## **1.3 Purpose of the Project**

The purpose of this project is to design and describe a machine learning system that detects potentially malicious insider activity by combining Elastic Net for feature selection with Random Forest for classification, and to present the result through an interactive web interface. The project aims to show how a two-stage pipeline can be built, evaluated honestly on imbalanced data, and interpreted by a security analyst.

## **1.4 Problem Statement**

Given a record of user activity described by behavioural attributes such as logins, file access, device usage, email and web activity, classify the activity as normal or a potential insider threat. The system must cope with a large number of features, some of which are redundant or irrelevant; handle the severe imbalance between normal and malicious records; model non-linear interactions between behaviours; avoid information leakage between training and testing; and provide results that an analyst can understand and act upon.

## **1.5 Objectives**

1. To study the insider-threat problem and the machine learning methods relevant to it.
2. To explore and pre-process user-activity data, including aggregation, encoding and scaling.
3. To select important features using Elastic Net regularisation.
4. To classify activity as normal or potential threat using a Random Forest.
5. To evaluate the model using accuracy, precision, recall, F1-score, ROC-AUC and PR-AUC, together with a confusion matrix.
6. To provide an interactive website for prediction, analysis and monitoring.
7. To discuss applications, limitations and future improvements.

## **1.6 Scope of the Project**

The scope covers the complete machine learning workflow for tabular behavioural data: data understanding, pre-processing, exploratory analysis, feature selection, model building, tuning, evaluation and interpretation, together with a web dashboard that presents the outputs. The project concentrates on supervised classification using Elastic Net and Random Forest; deep learning, graph-based and sequence models are discussed as future extensions. Integration with live enterprise logging systems is outside the present scope.

## **1.7 Importance of the Project**

The project applies well-established techniques to a practical and growing security problem. Elastic Net gives a transparent way to reduce the feature set, and Random Forest provides strong accuracy together with feature-importance values that help explain each decision. The combined approach is easier to justify to security managers than a black-box system, and it trains quickly on ordinary hardware.

## Chapter 2: Literature Survey

This chapter reviews the machine learning methods relevant to insider-threat detection and the research on user-behaviour analytics. The discussion moves from regularised linear models to tree-based ensembles, and then to insider-threat work and the CERT dataset, in order to identify the research gap addressed by this project.

### 2.1 Linear and Logistic Regression

Linear models describe the target as a weighted sum of input features. For a two-class problem such as normal versus malicious activity, Logistic Regression passes this weighted sum through a sigmoid function to obtain a probability [5]. It is simple and interpretable, but with many correlated predictors its coefficients become unstable and the model can overfit.

### 2.2 Ridge Regression

Ridge Regression, introduced by Hoerl and Kennard [3], adds an L2 penalty equal to the sum of squared coefficients. The penalty shrinks coefficients smoothly and stabilises them when features are correlated, but no coefficient becomes exactly zero, so Ridge performs no feature selection.

### 2.3 Lasso Regression

Lasso, proposed by Tibshirani [2], adds an L1 penalty equal to the sum of absolute coefficients. This can drive some coefficients exactly to zero and so selects features. Among a group of highly correlated features, however, Lasso tends to keep one arbitrarily and discard the others.

### 2.4 Elastic Net

Elastic Net, proposed by Zou and Hastie [1], combines the L1 and L2 penalties. It keeps the sparsity of Lasso while adding the grouping effect of Ridge, so that correlated features tend to be retained or removed together. Efficient coordinate-descent algorithms for fitting it are given by Friedman, Hastie and Tibshirani [9] and are available in scikit-learn [7]. These properties suit behavioural data, in which email count and attachment count, or after-hours activity and after-hours logins, are naturally correlated.

### 2.5 Decision Tree and Random Forest

A decision tree partitions the feature space with recursive splits and predicts the majority class in each region [10]. It captures non-linear effects and interactions but a single tree is unstable. Random Forest, introduced by Breiman [8], combines many trees, each trained on a bootstrap sample with a random subset of features at each split, and takes a majority vote. This reduces variance, handles interactions such as USB use combined with after-hours logon, and yields feature-importance measures. The price is reduced transparency compared with a linear model.

### 2.6 Insider Threat Research and the CERT Dataset

Insider-threat research distinguishes malicious insiders, who intentionally cause harm, from negligent insiders, whose mistakes expose data. Practical guidance from the CERT Insider Threat Center [11] describes recurring patterns such as data exfiltration through removable media and email, and unusual working hours. Because real incident data are confidential, researchers commonly use the CERT Insider Threat Dataset, a simulated corpus of enterprise activity generated by Glasser and Lindauer [12]. It contains logon, device, file, email and web records together with organisational information, and a small number of injected malicious scenarios.

## 2.7 Class Imbalance

In insider-threat data, malicious records form a tiny fraction of the total. A classifier that labels everything as normal can therefore achieve high accuracy while detecting nothing. Techniques such as class weighting and oversampling, including SMOTE [13], and evaluation with recall, F1-score and PR-AUC rather than accuracy alone, are standard remedies.

## 2.8 Comparison of Methods

**Table 2.1: Comparison of methods considered**

| Method              | Penalty         | Feature selection | Correlated features     | Non-linear | Interpretability |
|---------------------|-----------------|-------------------|-------------------------|------------|------------------|
| Logistic Regression | None            | No                | Unstable                | No         | High             |
| Ridge               | L2              | No                | Handled well            | No         | High             |
| Lasso               | L1              | Yes               | Picks one               | No         | High             |
| Elastic Net         | L1 + L2         | Yes               | Handled well (grouping) | No         | High             |
| Decision Tree       | Depth / pruning | Implicit          | Moderate                | Yes        | Medium           |
| Random Forest       | Ensembling      | Implicit          | Good                    | Yes        | Low–Medium       |

## 2.9 Research Gap

Many insider-threat studies apply a single classifier to all available features, which leaves noisy and redundant inputs in place, or report accuracy on imbalanced data, which hides poor detection of the rare malicious class. There is a need for a transparent two-stage pipeline in which regularised feature selection reduces the input space and a non-linear ensemble performs the final classification, evaluated with metrics appropriate to imbalance. This project addresses that gap.

## Chapter 3: Problem Analysis and Existing System

### 3.1 Existing Monitoring Methods

Organisations commonly monitor insiders through access-control lists, audit logs, data-loss-prevention tools and fixed alert rules. These methods record what happened and can flag events that cross a predefined limit, but they depend on analysts to review logs and on rules written in advance.

### 3.2 Manual Review

In manual review, security staff read logs or reports after an incident or at intervals. The approach depends on individual judgement, cannot cover the volume of events produced daily, is slow to respond and is rarely reproducible.

### 3.3 Rule-Based Detection

Rule-based systems alert when, for example, a user copies more than a set number of files or logs in at night. Rules treat each behaviour separately, ignore differences between roles and are easily avoided by an insider who stays just below the threshold.

### 3.4 Problems and Disadvantages of Existing Methods

- Reactive operation: suspicious behaviour is found only after damage has occurred.
- Volume: large amounts of activity data make manual monitoring impractical.
- False alarms: fixed thresholds ignore the normal level of each user or role.
- Ignored interactions: file, device, email and logon behaviour are examined in isolation.
- Noise: irrelevant or redundant features reduce the quality of any learned model.
- Imbalance: accuracy-based evaluation hides failure to detect the rare malicious class.

### 3.5 Target Users

The intended users are security operations centre analysts, IT administrators, compliance and audit teams, and managers responsible for protecting sensitive information.

### 3.6 Proposed Improvement

**Table 3.1: Existing approach versus proposed approach**

| Aspect              | Existing approach              | Proposed approach                                     |
|---------------------|--------------------------------|-------------------------------------------------------|
| Decision style      | Reactive and manual            | Predictive and data-driven                            |
| Use of data         | Logs reviewed after the event  | Learns from historical user activity                  |
| Feature handling    | Single thresholds in isolation | Combines logon, file, device, email and web behaviour |
| Feature selection   | Manual judgement               | Automatic through the Elastic Net L1 penalty          |
| Non-linear patterns | Not captured                   | Captured by Random Forest                             |

| Aspect       | Existing approach | Proposed approach                                  |
|--------------|-------------------|----------------------------------------------------|
| Evaluation   | Rarely validated  | Cross-validation and imbalance-aware metrics       |
| Presentation | Raw logs          | Interactive dashboard with alerts and explanations |

## Chapter 4: Proposed System

### 4.1 Proposed Insider Threat Detection System

The proposed system is a supervised classification pipeline that accepts user-activity records, converts them into behavioural features, selects the informative features with Elastic Net, classifies activity with Random Forest, and presents predictions and evaluation results in a web dashboard. The model pipeline is implemented in Python using scikit-learn [7], with every fitted transformation learned on the training data only.

### 4.2 System Architecture

The architecture has five layers. The data layer holds the activity sources (logon, device, file, email, web and organisational records). The processing layer performs cleaning, time-feature extraction, aggregation, encoding and scaling. The feature-selection layer applies Elastic Net. The model layer contains the Random Forest classifier, the baseline models and the tuning procedure. The presentation layer is the Insider Guard web application, which shows predictions, risk levels, alerts and analytics.

### 4.3 System Workflow

**Figure 4.1: Workflow of the proposed system**

Dataset → Data Preprocessing → Feature Engineering → Train / Test Split → Elastic Net Feature Selection → Random Forest Classification → Threat Prediction → Evaluation and Website

Raw activity data are loaded and inspected. Pre-processing removes duplicates, treats missing values, encodes categorical variables and scales numerical variables. Feature engineering aggregates events into user-level or user-period features and derives indicators such as after-hours activity. The data are split into training and test sets, Elastic Net selects the important features on the training data, and Random Forest is trained on the selected features. The model predicts normal activity or potential threat for new records and is evaluated on the held-out test set.

### 4.4 Functional Requirements

- The system shall load user-activity data from a structured file such as CSV.
- The system shall report missing and duplicate values and validate data types.
- The system shall scale numerical features and encode categorical features.
- The system shall select features using Elastic Net and display the retained and removed features.
- The system shall classify activity as normal or potential threat using Random Forest and report a risk probability and risk level.
- The system shall report accuracy, precision, recall, F1-score, ROC-AUC, a confusion matrix and feature importance.

- The system shall provide a login, dashboard, user-activity view, alerts, prediction history and reports.

#### 4.5 Non-Functional Requirements

- Accuracy: predictions should generalise to unseen data and be evaluated honestly.
- Interpretability: selected features and importance values should explain each prediction.
- Performance: training and prediction should complete quickly on a standard computer.
- Reproducibility: a fixed random state should give repeatable results.
- Security and privacy: user identifiers and activity data should be handled with access control.
- Maintainability: the pipeline should be easy to retrain when new data arrive.

#### 4.6 Hardware and Software Requirements

**Table 4.1: Hardware and software requirements**

| Category             | Requirement                                                   |
|----------------------|---------------------------------------------------------------|
| Processor            | Standard multi-core CPU; no GPU required                      |
| Memory               | Minimum 4 GB RAM (8 GB recommended)                           |
| Storage              | Few hundred MB for data, notebooks and output                 |
| Operating system     | Windows or Linux                                              |
| Programming language | Python 3 (model); TypeScript / React (web interface)          |
| Libraries            | Pandas, NumPy, Scikit-learn, Matplotlib                       |
| Environment          | Jupyter Notebook or VS Code; Node.js to run the web interface |
| Dataset              | Insider-threat / user-behaviour activity dataset (CERT)       |

## Chapter 5: Dataset and Data Preprocessing

### 5.1 Dataset Description

The project is designed around the CERT Insider Threat Dataset [12], a simulated dataset of enterprise user activity organised by source: logon, device, file, email, HTTP/web, LDAP and psychometric records. The exact number of records, columns, class counts and missing values are those of the final dataset used in the implementation and are to be read directly from it when the notebook is executed; they are not stated here so that no figure is invented. The behavioural features derived from the raw sources are listed in Table 5.1.

**Table 5.1: Behavioural features**

| Feature                | Purpose                                         | Role   |
|------------------------|-------------------------------------------------|--------|
| login_count            | Total login activity during the selected period | Input  |
| after_hours_logins     | Login activity outside normal working hours     | Input  |
| failed_logins          | Unsuccessful login attempts, if available       | Input  |
| file_access_count      | Number of file-access activities                | Input  |
| file_copy_count        | Number of file-copy activities, if available    | Input  |
| usb_usage              | Device / USB activity                           | Input  |
| email_count            | Number of email activities                      | Input  |
| external_email_count   | Email involving external recipients             | Input  |
| email_attachment_count | Number of email attachments                     | Input  |
| web_activity_count     | Number of web / HTTP activities                 | Input  |
| unique_domains         | Number of distinct web domains visited          | Input  |
| after_hours_activity   | Total activity outside normal hours             | Input  |
| department / role      | Organisational context, where available         | Input  |
| threat                 | Normal (0) or suspicious (1) behaviour          | Target |

### 5.2 Data Types

Counts such as logins and file accesses are numerical; department and role are categorical; raw event times are timestamps. Each column is checked for the correct data type before processing.

### 5.3 Missing Values

Missing values are counted for every feature with `df.isnull().sum()`. Numerical columns are filled with the median, which is robust to extreme values, and categorical columns with the most frequent category or an explicit unknown label. Columns with excessive or irrelevant missingness may be removed after inspection.

### 5.4 Duplicate Records

Duplicate records are identified with `df.duplicated().sum()` and removed where they represent genuine duplicate events, because repeated rows overweight some observations and can appear in both training and test sets, making evaluation optimistic.

## **5.5 Time Processing and Feature Aggregation**

Timestamps are converted into hour of day, day of week and an after-hours indicator. Raw events are then aggregated into user-level or user-period features, since the classifier works on one row of behaviour per user and period rather than on individual events.

## **5.6 Outlier Checking**

Box plots and the inter-quartile range (IQR) rule are used to flag unusually high or low activity. In this project an extreme value is not automatically an error, because unusual behaviour may itself be the signal of a threat. Flagged values are therefore investigated before any decision is made to remove, transform or retain them; only impossible values are corrected.

## **5.7 Categorical Encoding**

Department and role are one-hot encoded, creating a 0/1 column for each category, with unknown categories ignored at prediction time.

## **5.8 Numerical Scaling**

Regularisation penalises the size of coefficients, so features on large scales would otherwise be penalised unfairly. Numerical features are standardised to zero mean and unit variance using StandardScaler, fitted on the training data only. Scaling is needed for Elastic Net; Random Forest is insensitive to it.

## **5.9 Handling Class Imbalance**

Malicious records are expected to be a small minority. The class distribution is therefore examined first, a stratified split keeps the same class proportions in training and test sets, and the Random Forest uses balanced class weights. Oversampling such as SMOTE [13] may be applied to the training set only, never to the test set.

## **5.10 Train-Test Split and Prevention of Leakage**

The data are divided into a training set, used to fit and tune the models, and a test set, used once for the final evaluation, for example in an 80/20 stratified split with a fixed random state. Because user activity is ordered in time, a chronological split may give a more realistic estimate of future performance. Information leakage is prevented by fitting the scaler, the Elastic Net selector and any resampling on the training data only, and by excluding any feature that directly restates the label, such as an investigation outcome or risk score assigned after the event. All fitted transformations are applied to the test data afterwards.

## Chapter 6: Exploratory Data Analysis

Exploratory Data Analysis (EDA) examines the data visually and statistically before modelling in order to understand its structure and check that the assumptions of the models are reasonable. The patterns below are those expected in user-activity data and are to be confirmed against the actual dataset; no numerical findings are claimed here.

### 6.1 Data Loading and Initial Inspection

The dataset is loaded into a Pandas DataFrame and inspected with `df.head()`, `df.shape`, `df.columns`, `df.info()` and `df.describe()` to establish its dimensions, column names, data types and non-null counts.

### 6.2 Descriptive Statistics and Univariate Analysis

Mean, standard deviation, minimum, maximum, median and quartiles describe the centre and spread of each behavioural variable. Histograms and bar charts of login count, file activity, email activity, USB usage and web activity reveal skewness and unusual ranges. Activity counts are expected to be strongly skewed because a small number of users generate many events, which may call for transformation or careful interpretation.

### 6.3 Outlier Analysis

Box plots identify unusually high or low activity. As noted in Section 5.6, outliers are investigated rather than deleted automatically, since they may carry the very information the model needs.

### 6.4 Correlation Analysis

A correlation heatmap reveals redundant or strongly related variables; for example, email count may relate to attachment count, and after-hours activity to after-hours logins. Strongly correlated predictors are the situation in which Elastic Net is expected to be more stable than Lasso. Correlation supports feature selection and interpretation but does not establish causation.

### 6.5 Class Distribution Analysis

The target is examined with a frequency table and bar chart. Normal activity is expected to far outnumber suspicious activity. If the data are imbalanced, evaluation must not rely on accuracy alone; precision, recall, F1-score, the confusion matrix, ROC-AUC and PR-AUC are reported as well.

### 6.6 Feature versus Target Analysis

Behavioural features are compared across the target classes with box plots and grouped bar charts, for example file access, after-hours activity, external email activity and USB activity against the threat class. These comparisons show which features may carry predictive information.

## 6.7 EDA Findings and Interpretation

8. User activity is heterogeneous; different users have substantially different activity patterns.
9. Activity-count features may be skewed because a few users generate many events.
10. After-hours activity can provide useful behavioural information.
11. File, email, device, login and web activity all contribute to a user-behaviour profile.
12. Potential class imbalance must be checked before model training.
13. Highly correlated features should be reviewed to reduce redundancy.
14. Unusual values should be investigated rather than automatically deleted.
15. Raw event data should be converted into aggregated behavioural features before classification.

## 6.8 Recommended Graphs and Charts

**Table 6.1: Recommended graphs for exploratory analysis**

| Graph                 | Variables                | Purpose                       |
|-----------------------|--------------------------|-------------------------------|
| Feature summary table | All columns              | Show structure and data types |
| Missing-value summary | All columns              | Locate incomplete data        |
| Bar chart             | Target class             | Show class imbalance          |
| Histograms            | Major numerical features | Check distribution and skew   |
| Box plots             | Activity-count features  | Detect outliers               |
| Correlation heatmap   | All numerical features   | Detect redundancy             |
| Box plots by class    | Feature vs threat        | Find discriminating features  |

## Chapter 7: Elastic Net and Random Forest Methodology

### 7.1 Classification

Classification is a supervised learning task that assigns an input to one of a set of classes. Here the two classes are normal activity (0) and potential threat (1), predicted from the behavioural features of a user.

### 7.2 Why a Two-Stage Approach

Elastic Net and Random Forest have complementary strengths. Elastic Net handles many correlated features and removes uninformative ones, producing a smaller and cleaner input set. Random Forest then models non-linear relationships and interactions among the retained features, which a linear model cannot. Together they give a compact, accurate and explainable pipeline.

### 7.3 Elastic Net

Elastic Net applies the L1 and L2 penalties simultaneously. For a linear model the coefficients are found by minimising the following objective function.

$$\text{Minimise: } (1/(2n)) \|y - X\beta\|^2 + \alpha [l1\_ratio \|\beta\|_1 + ((1 - l1\_ratio)/2) \|\beta\|^2]$$

When the target is a class label, the same penalty is added to the logistic loss (an elastic-net penalised logistic regression). The symbols are explained in Table 7.1.

**Table 7.1: Symbols used in the Elastic Net objective**

| Symbol             | Meaning                                                                    |
|--------------------|----------------------------------------------------------------------------|
| n                  | Number of training observations (rows)                                     |
| y                  | Vector of actual target values                                             |
| X                  | Matrix of input features, one row per observation                          |
| $\beta$            | Vector of model coefficients to be learned                                 |
| $\ y - X\beta\ ^2$ | Sum of squared prediction errors                                           |
| $\alpha$ (alpha)   | Overall strength of regularisation; larger values shrink coefficients more |
| l1_ratio           | Mixing parameter between 0 and 1 that sets the share of the L1 penalty     |
| $\ \beta\ _1$      | L1 norm: sum of absolute values of the coefficients                        |
| $\ \beta\ ^2$      | Squared L2 norm: sum of squared coefficients                               |

### 7.4 L1 and L2 Regularisation

L1 regularisation penalises the absolute size of coefficients and creates sparsity, so weak features receive a coefficient of exactly zero. L2 regularisation penalises the squared size and spreads weight across correlated features, making the solution stable. Elastic Net uses both, and the balance is set by l1\_ratio: a value of 1 gives pure Lasso and a value near 0 behaves like Ridge.

### 7.5 Feature Selection with Elastic Net

After scaling, the model is fitted on the training data. Features whose coefficients are non-zero are retained; those shrunk to zero are removed. The retained set is passed to the Random Forest. The selection is made on the training data only so that no information from the test set influences it.

## **7.6 Hyperparameter Tuning**

Alpha and l1\_ratio are not learned from the data. They are tuned by a grid of candidate values, with alpha on a logarithmic scale and several l1\_ratio values between 0 and 1, selecting the combination with the best cross-validated score. In scikit-learn this is done with GridSearchCV or the CV variants of the linear models [7].

## **7.7 Random Forest**

Random Forest builds many decision trees, each on a bootstrap sample of the training data and using a random subset of features at each split. Each tree votes, and the majority class is the prediction; the share of votes gives a risk probability. Because trees split on thresholds, they can capture interactions such as high file access combined with USB use outside working hours. The main parameters are the number of trees, the maximum depth, the minimum samples per split and the class weights. Feature importance, computed from the reduction in impurity that each feature provides, indicates which behaviours drive the decision.

## **7.8 Cross-Validation**

In stratified k-fold cross-validation the training data are divided into k parts that keep the class proportions. The model is trained on  $k - 1$  parts and validated on the remaining one, and this is repeated until every part has been used for validation. The average score is a more reliable estimate than a single split. For time-ordered data a time-series cross-validation that always validates on later records is preferable.

## Chapter 8: Model Development and Evaluation

### 8.1 Model Training Process

The steps are: load and clean the data; aggregate and engineer features; split into training and test sets with stratification; fit the encoder and scaler on the training data; run Elastic Net to select features; train the Random Forest on the selected features; tune hyperparameters by cross-validation; predict on the test set; and compute the evaluation metrics. The same random state is used for all models so that the comparison is fair.

### 8.2 Baseline Models

To judge whether the proposed pipeline is worthwhile, it is compared with a standalone Random Forest trained on all features, an Elastic Net logistic regression used alone, and, optionally, a single decision tree and a Support Vector Machine. Any more complex model must outperform these simple benchmarks to be justified.

### 8.3 Evaluation Metrics

**Table 8.1: Classification evaluation metrics**

| Metric    | Formula                           | Meaning                                                                  |
|-----------|-----------------------------------|--------------------------------------------------------------------------|
| Accuracy  | $(TP + TN) / (TP + TN + FP + FN)$ | Share of all predictions that are correct; misleading on imbalanced data |
| Precision | $TP / (TP + FP)$                  | Of the users flagged as threats, the share that truly are                |
| Recall    | $TP / (TP + FN)$                  | Of the real threats, the share that were detected                        |
| F1-score  | $2 \times P \times R / (P + R)$   | Harmonic mean of precision and recall                                    |
| ROC-AUC   | Area under TPR vs FPR curve       | Ability to rank threats above normal users at all thresholds             |
| PR-AUC    | Area under precision-recall curve | Preferred measure when the positive class is rare                        |

Here TP is a threat correctly flagged, TN is normal activity correctly passed, FP is a normal user wrongly flagged and FN is a threat that was missed. In security monitoring, a missed threat (FN) is usually more costly than a false alarm, so recall is given particular weight, while precision limits analyst workload.

### 8.4 Confusion Matrix

**Table 8.2: Confusion matrix layout (values from the final test set)**

|               | Predicted Normal                | Predicted Threat                |
|---------------|---------------------------------|---------------------------------|
| Actual Normal | TN: To be filled from final run | FP: To be filled from final run |
| Actual Threat | FN: To be filled from final run | TP: To be filled from final run |

### 8.5 Model Comparison

**Table 8.3: Comparison of models on the test set**

| Model                                  | Accuracy | Precision | Recall | F1  | ROC-AUC | PR-AUC |
|----------------------------------------|----------|-----------|--------|-----|---------|--------|
| Elastic Net + Random Forest (proposed) | TBD      | TBD       | TBD    | TBD | TBD     | TBD    |
| Random Forest (all features)           | TBD      | TBD       | TBD    | TBD | TBD     | TBD    |
| Elastic Net logistic regression        | TBD      | TBD       | TBD    | TBD | TBD     | TBD    |
| Decision Tree                          | TBD      | TBD       | TBD    | TBD | TBD     | TBD    |
| SVM (RBF kernel)                       | TBD      | TBD       | TBD    | TBD | TBD     | TBD    |

The values marked TBD can be obtained only by running the models on the actual dataset and are intentionally left blank. No performance figure has been assumed or invented in this report.

## Chapter 9: Results and Discussion

This chapter explains how the experimental results are to be analysed and interpreted. Because the numerical results are calculated from the actual dataset, the discussion describes the expected behaviour and the checks to be performed rather than quoting values.

### 9.1 Features Selected by Elastic Net

The coefficients learned by Elastic Net are tabulated, with retained and removed features identified. Behaviours expected to be retained include file access, USB or device usage, after-hours activity, external email activity and failed logins, since these correspond to known exfiltration patterns [11]. The final list must be taken from the trained model.

**Table 9.1: Elastic Net coefficients and Random Forest importance (from final run)**

| Feature        | Coefficient | Status | RF importance |
|----------------|-------------|--------|---------------|
| (to be filled) | —           | —      | —             |
| (to be filled) | —           | —      | —             |
| (to be filled) | —           | —      | —             |

### 9.2 Random Forest Classification Results

Accuracy, precision, recall, F1-score, ROC-AUC and PR-AUC are reported for the test set, together with the confusion matrix, the ROC curve and the precision-recall curve. The analysis should state how many real threats were detected, how many normal users were falsely flagged, and what this means for analyst workload.

### 9.3 Effect of Feature Selection

Comparing the proposed pipeline with a Random Forest on all features shows whether Elastic Net improved accuracy, reduced training time or simplified the model. Where results are similar, the smaller feature set is still an advantage because it is easier to interpret and cheaper to collect.

### 9.4 Feature Importance and Interpretation

Random Forest importance values indicate which behaviours influence the decision most. A prediction is explained to the analyst by showing the contributing behaviours, such as high file access together with USB use after hours, so that a risk score can be investigated rather than accepted blindly.

### 9.5 Threshold Selection

The default threshold of 0.5 on the predicted probability is not necessarily optimal for rare events. The threshold can be chosen from the precision-recall curve to reach a target recall while keeping false alarms manageable, and the risk level (low, medium, high) can be defined from probability bands.

### 9.6 Factors Affecting Prediction

**Table 9.2: Factors that may affect prediction quality**

| Factor                | Effect on prediction                                                      |
|-----------------------|---------------------------------------------------------------------------|
| Data quality          | Missing, duplicated or wrongly recorded events reduce accuracy            |
| Class imbalance       | Rare threats may be missed unless weighting and suitable metrics are used |
| Simulated data        | Models trained on simulated data may not transfer to real organisations   |
| Changing behaviour    | Role changes and new work patterns create drift and false alarms          |
| Hyperparameter choice | Too much regularisation drops useful features; too little keeps noise     |
| Information leakage   | Gives unrealistically good test results that fail in real use             |
| Intentional evasion   | Insiders who stay close to normal behaviour are harder to detect          |

## Chapter 10: Web Application (Insider Guard)

### 10.1 Purpose

Insider Guard is an interactive web dashboard that presents the outputs of the pipeline to a security analyst. It is built with React and TypeScript using the Vite build tool and Tailwind CSS, and runs in a standard browser.

### 10.2 Modules

**Table 10.1: Main sections of the interface**

| Section                        | Description                                                                                                                                |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Login                          | Authenticates the analyst before the dashboard is shown                                                                                    |
| Dashboard                      | Summary of users, threats, risk levels and recent alerts                                                                                   |
| User Activity                  | Per-user behavioural records and search                                                                                                    |
| Threat Detection               | Form for entering activity values and obtaining a prediction, risk probability, risk level, contributing features and a recommended action |
| Alerts                         | List of generated alerts with status and key anomalies                                                                                     |
| Analytics                      | Threat distribution, normal versus threat activity, risk-level distribution and correlation heatmap                                        |
| Elastic Net / Random Forest    | Feature coefficients, retained and removed features, and importance values                                                                 |
| Model Evaluation               | Metrics, confusion matrix and comparison with baseline models                                                                              |
| Monitoring and Retraining      | Prediction volume, threat rate, drift indicators and model refresh                                                                         |
| Prediction History and Reports | Log of past predictions and downloadable summaries                                                                                         |
| Raw Dataset                    | Tabular view and CSV export of the records                                                                                                 |

### 10.3 Prediction Flow

The analyst opens the Threat Detection page and enters login frequency, file access count, sensitive files accessed, email and external-recipient counts, USB use, after-hours activity, failed logins and data transferred. The values are standardised, the retained features are weighted, the ensemble returns a risk probability, and the page displays the label, risk level (Low below 40%, Medium from 40%, High from 75%), the contributing behaviours and a recommended action.

### 10.4 MLOps and Deployment

**Figure 10.1: MLOps cycle**

Data → Training → Saved Model → Prediction API → Website → Monitoring → Retraining

The trained model is saved and served through a prediction interface used by the website. Monitoring tracks prediction volume, latency, threat rate and data drift, and triggers retraining when the data change.

### 10.5 Implementation Status

In its present form the web interface demonstrates the workflow with a seeded, generated demonstration dataset and an inference module that reproduces the behaviour of the pipeline in the browser. The coefficients, risk probabilities and comparison figures shown in the interface are therefore illustrative. For the final submission the model should be trained on the actual CERT-based dataset in Python, and the real coefficients, feature importances and metrics should replace the demonstration values in both the interface and Chapters 8 and 9.

## **Chapter 11: Applications, Advantages and Limitations**

### **11.1 Applications**

- Enterprise security: early warning of data theft by employees and contractors.
- Banking and finance: monitoring access to customer and transaction data.
- Healthcare: detecting inappropriate access to patient records.
- Government and defence: protecting classified information.
- Education and research: protecting examination material and research data.
- Security operations centres: prioritising alerts for analyst review.

### **11.2 Advantages**

- Elastic Net removes irrelevant features and handles correlated ones.
- Random Forest captures non-linear interactions and resists overfitting.
- Feature importance and coefficients make results explainable.
- Trains quickly on ordinary hardware and is easy to retrain.
- The dashboard gives analysts a clear risk level and recommended action.

### **11.3 Limitations**

- Depends on the quality and realism of the data; simulated data may not reflect real organisations.
- Rare malicious cases make detection and evaluation difficult.
- A flagged user is not proof of malice; human review is required.
- Monitoring user activity raises privacy and ethical concerns that policy must address.
- Models may need retraining as behaviour and roles change.

## Conclusion

This project presented the design of an insider-threat detection system that combines Elastic Net feature selection with Random Forest classification. Insider activity is difficult to distinguish from normal work, and the volume of data makes manual monitoring impractical. Elastic Net addresses the large and correlated feature space by retaining only informative behaviours, and Random Forest models the non-linear interactions among those behaviours to classify activity as normal or a potential threat. A complete workflow was described, covering exploratory analysis, cleaning, feature engineering, scaling, leakage prevention, handling of class imbalance, tuning by cross-validation and evaluation with accuracy, precision, recall, F1-score, ROC-AUC and PR-AUC. The accompanying Insider Guard dashboard presents predictions, alerts and analytics in an accessible form. Used with a carefully prepared dataset and evaluated with imbalance-aware metrics, the approach can give security teams an accurate, explainable and economical way to identify suspicious activity at an early stage.

## Future Scope

- Real-time activity monitoring through integration with enterprise logging and SIEM systems.
- Larger and more diverse datasets, including real organisational data where permitted.
- Advanced anomaly detection and unsupervised methods for previously unseen attack patterns.
- Sequence and deep learning models, such as LSTM networks, to capture behaviour over time.
- Gradient boosting methods such as XGBoost for comparison with Random Forest.
- Automated model retraining with drift detection.
- Explainability tools such as SHAP for per-prediction explanations.
- Privacy-preserving techniques so that monitoring respects user rights.

## References

- [1] Zou, H., & Hastie, T. (2005). Regularization and variable selection via the elastic net. *Journal of the Royal Statistical Society: Series B*, 67(2), 301–320.
- [2] Tibshirani, R. (1996). Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B*, 58(1), 267–288.
- [3] Hoerl, A. E., & Kennard, R. W. (1970). Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1), 55–67.
- [4] Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning* (2nd ed.). Springer.
- [5] James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An Introduction to Statistical Learning with Applications in R* (2nd ed.). Springer.
- [6] Scikit-learn developers. Linear Models: Ridge, Lasso and Elastic Net. Scikit-learn User Guide. [https://scikit-learn.org/stable/modules/linear\\_model.html](https://scikit-learn.org/stable/modules/linear_model.html)
- [7] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- [8] Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
- [9] Friedman, J., Hastie, T., & Tibshirani, R. (2010). Regularization paths for generalized linear models via coordinate descent. *Journal of Statistical Software*, 33(1), 1–22.
- [10] Breiman, L., Friedman, J., Olshen, R., & Stone, C. (1984). *Classification and Regression Trees*. Wadsworth.
- [11] Cappelli, D., Moore, A., & Trzeciak, R. (2012). *The CERT Guide to Insider Threats*. Addison-Wesley.
- [12] Glasser, J., & Lindauer, B. (2013). Bridging the gap: A pragmatic approach to generating insider threat data. *IEEE Security and Privacy Workshops*, 98–104.
- [13] Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321–357.